Visualización con Matplotlib — Olist

0. Hacer tu guía

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (10, 5)
plt.rcParams["axes.grid"] = True

orders = pd.read_csv("olist_orders_dataset.csv")
order_items = pd.read_csv("olist_order_items_dataset.csv")
customers = pd.read_csv("olist_customers_dataset.csv")
products = pd.read_csv("olist_products_dataset.csv")
sellers = pd.read_csv("olist_sellers_dataset.csv")
order_payments = pd.read_csv("olist_order_payments_dataset.csv")
order_reviews = pd.read_csv("olist_order_reviews_dataset.csv")
translation = pd.read_csv("product_category_name_translation.csv")

print("orders:", orders.shape)
print("order_items:", order_items.shape)
print("customers:", customers.shape)
print("products:", products.shape)
print("order_payments:", order_payments.shape)
print("order_reviews:", order_reviews.shape)

orders.head()
```

```
orders: (99441, 8)
order_items: (112650, 7)
customers: (99441, 5)
products: (32951, 9)
order_payments: (103886, 5)
order_reviews: (99224, 7)
```

Out[1]:

	order_id	customer_id	order_st
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	deliv
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	deliv
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	deliv
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	deliv
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	deliv

1. Línea — pedidos por mes

Paso a paso

1. Convertimos la fecha de compra a formato fecha.
2. Filtramos los pedidos entregados.
3. Creamos el mes y contamos pedidos únicos.
4. Quitamos los meses extremos con muy pocos datos.
5. Graficamos.

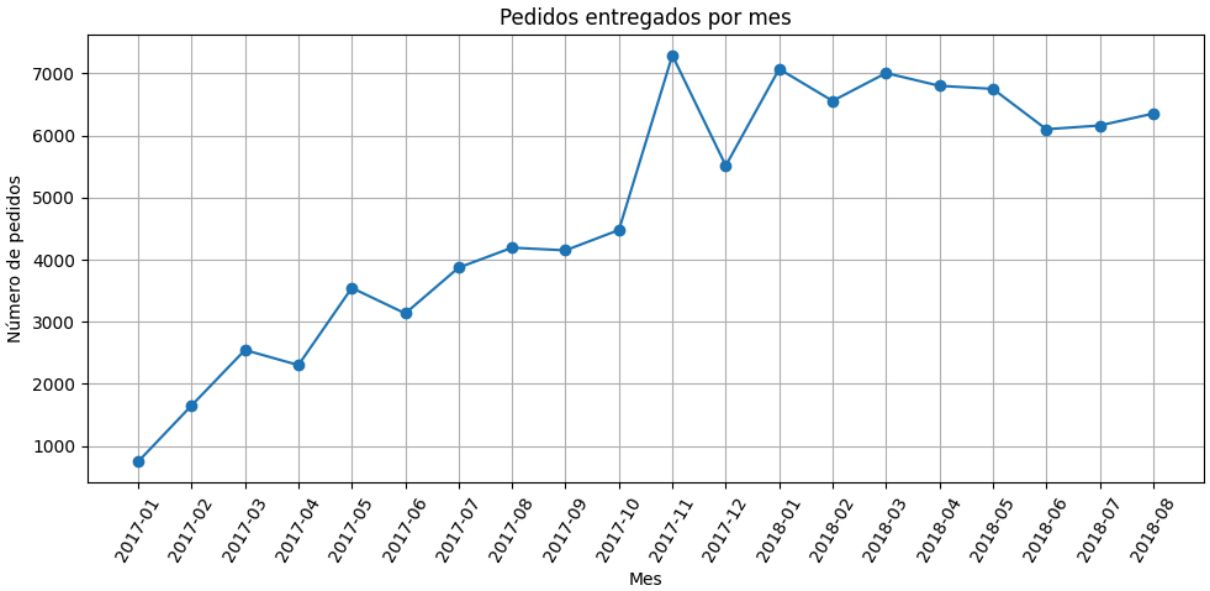
In [2]:
```
orders["order_purchase_timestamp"] = pd.to_datetime(
    orders["order_purchase_timestamp"], errors="coerce"
)

orders_delivered = orders[orders["order_status"] == "delivered"].copy()
orders_delivered["mes"] = orders_delivered["order_purchase_timestamp"].dt.to

pedidos_mes = orders_delivered.groupby("mes")["order_id"].nunique()

# Excluir los meses extremos: 2016 y septiembre-octubre de 2018
pedidos_mes = pedidos_mes[
    (pedidos_mes.index >= pd.Period("2017-01", freq="M")) &
    (pedidos_mes.index <= pd.Period("2018-08", freq="M"))
]

plt.plot(pedidos_mes.index.astype(str), pedidos_mes.values, marker="o")
plt.title("Pedidos entregados por mes")
plt.xlabel("Mes")
plt.ylabel("Número de pedidos")
plt.xticks(rotation=60)
plt.tight_layout()
plt.show()
```



2. Barras horizontales — top 10 categorías por ingreso

Paso a paso

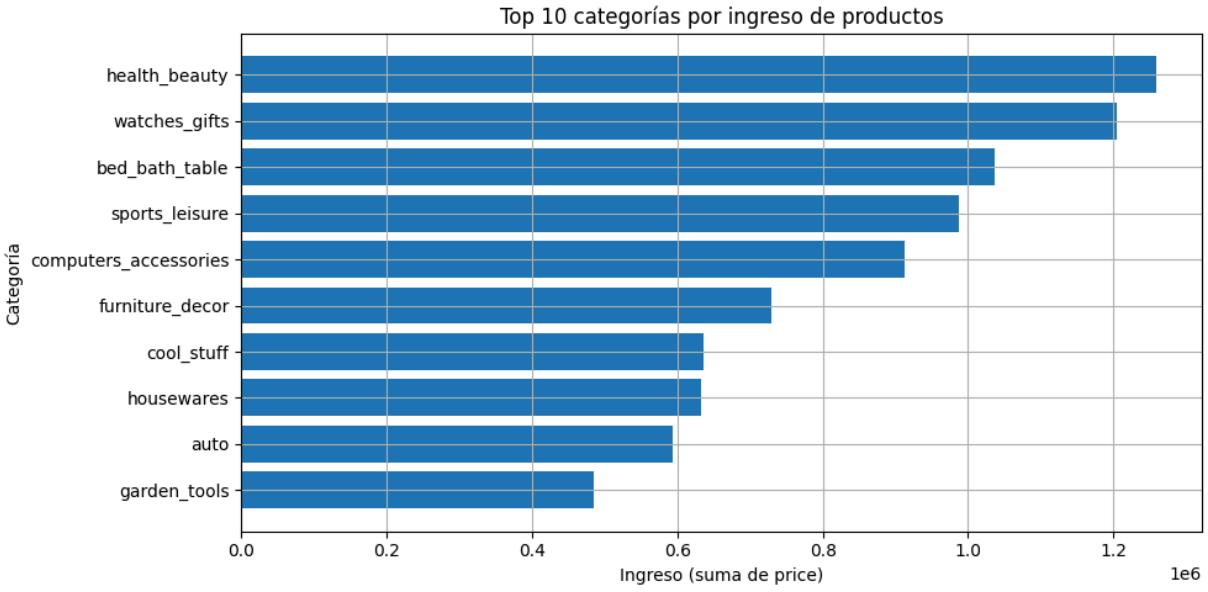
1. Unimos los artículos con los productos.
2. Unimos la traducción para mostrar las categorías en inglés.
3. Sumamos `price` por categoría.
4. Seleccionamos las 10 categorías con más ingreso y las dibujamos con `barh`.

In [3]:
```
# Buscamos la traducción de las categorías de productos
translation = pd.read_csv("product_category_name_translation.csv")
items_productos = order_items.merge(products, on="product_id", how="left")
items_categorias = items_productos.merge(translation, on="product_category_r
)

col_categoria = "product_category_name_english"
items_categorias[col_categoria] = items_categorias[col_categoria].fillna(
    items_categorias["product_category_name"]
).fillna("sin categoría")

ingreso_categoria = items_categorias.groupby(col_categoria)["price"].sum()
top10 = ingreso_categoria.nlargest(10).sort_values()

plt.barh(top10.index, top10.values)
plt.title("Top 10 categorías por ingreso de productos")
plt.xlabel("Ingreso (suma de price)")
plt.ylabel("Categoría")
plt.tight_layout()
plt.show()
```



3. Histograma — distribución del precio

Primero vemos los precios tal como están. Después usamos una escala logarítmica para observar mejor los precios pequeños y grandes. La media puede verse afectada por unos cuantos productos muy caros; por eso también marcamos la mediana.

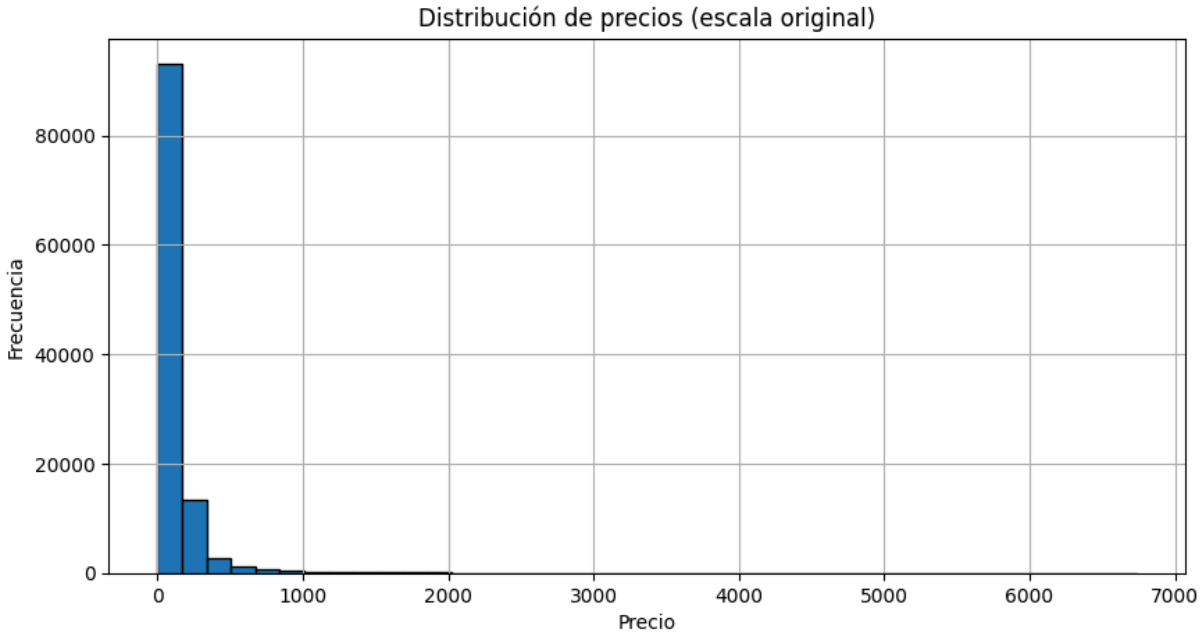
In [4]:
```
precios = order_items["price"].dropna()
precios_positivos = precios[precios > 0]

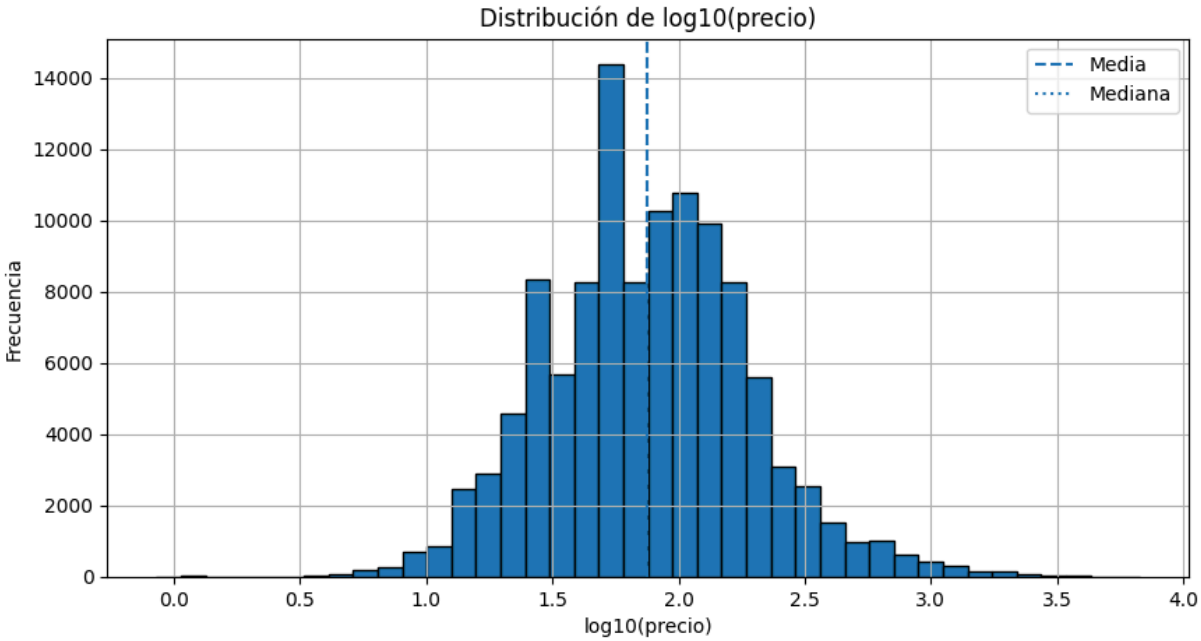
plt.hist(precios, bins=40, edgecolor="black")
plt.title("Distribución de precios (escala original)")
plt.xlabel("Precio")
plt.ylabel("Frecuencia")
plt.show()

log_precios = np.log10(precios_positivos)
media_log = log_precios.mean()
mediana_log = log_precios.median()

plt.hist(log_precios, bins=40, edgecolor="black")
plt.axvline(media_log, linestyle="--", label="Media")
plt.axvline(mediana_log, linestyle=":", label="Mediana")
plt.title("Distribución de log10(precio)")
plt.xlabel("log10(precio)")
plt.ylabel("Frecuencia")
plt.legend()
plt.show()

print("Media del precio original:", round(precios.mean(), 2))
print("Mediana del precio original:", round(precios.median(), 2))
```






```
Media del precio original: 120.65
Mediana del precio original: 74.99
```

4. Histograma comparativo — días reales vs. estimados de entrega

Calculamos los días desde la compra hasta la entrega real y hasta la fecha estimada. Se eliminan los pedidos sin entrega real para que la comparación use pedidos entregados.

In [5]:
```
columnas_fecha = [
    "order_purchase_timestamp",
    "order_delivered_customer_date",
    "order_estimated_delivery_date"
]

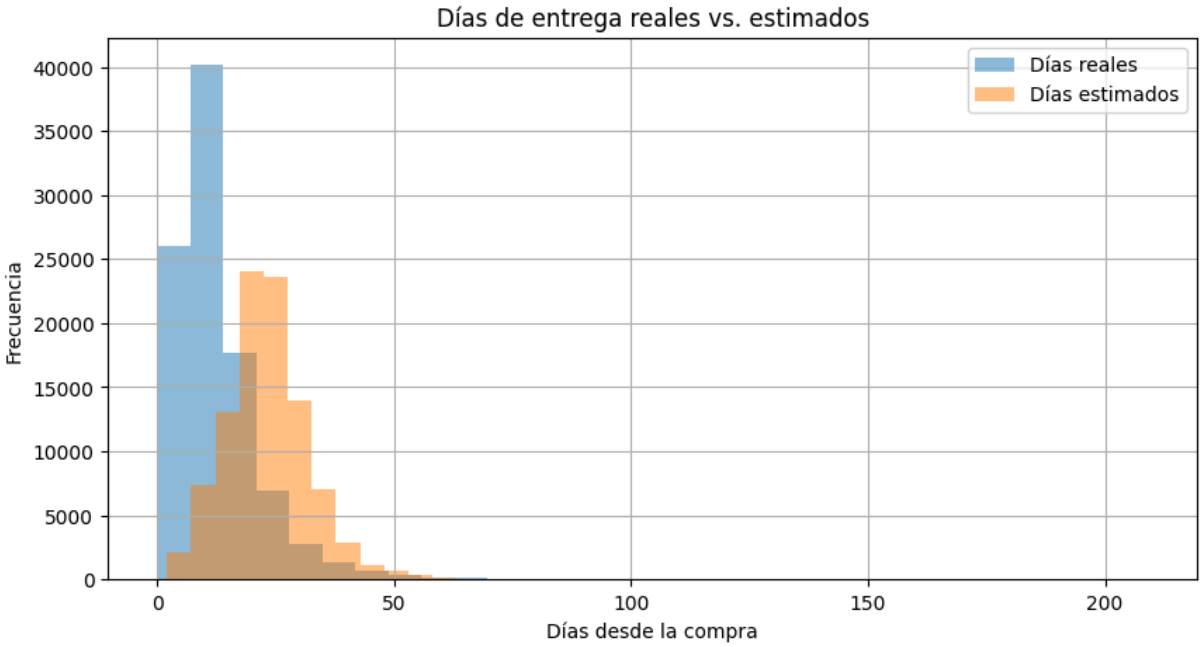
for columna in columnas_fecha:
    orders[columna] = pd.to_datetime(orders[columna], errors="coerce")

entregas = orders.dropna(subset=columnas_fecha).copy()
entregas["dias_reales"] = (
    entregas["order_delivered_customer_date"] -
    entregas["order_purchase_timestamp"]
).dt.days
entregas["dias_estimados"] = (
    entregas["order_estimated_delivery_date"] -
    entregas["order_purchase_timestamp"]
).dt.days

entregas = entregas[
    (entregas["dias_reales"] >= 0) &
    (entregas["dias_estimados"] >= 0)
]

plt.hist(entregas["dias_reales"], bins=30, alpha=0.5, label="Días reales")
```

```
plt.hist(entregas["dias_estimados"], bins=30, alpha=0.5, label="Días estimac
plt.title("Días de entrega reales vs. estimados")
plt.xlabel("Días desde la compra")
plt.ylabel("Frecuencia")
plt.legend()
plt.show()
```



5. Boxplot — calificación vs. días de retraso

El retraso es la fecha de entrega real menos la fecha estimada. Un valor positivo significa que llegó tarde; uno negativo, que llegó antes. Quitamos reseñas repetidas por pedido para evitar que un pedido tenga peso extra.

In [6]:
```
retrasos = orders.dropna(
    subset=["order_delivered_customer_date", "order_estimated_delivery_date"
).copy()

retrasos["retraso"] = (
    retrasos["order_delivered_customer_date"] -
    retrasos["order_estimated_delivery_date"]
).dt.days

resenas_unicas = order_reviews.drop_duplicates(subset="order_id", keep="firs
retraso_resena = retrasos.merge(
    resenas_unicas[["order_id", "review_score"]], on="order_id", how="inner"
)

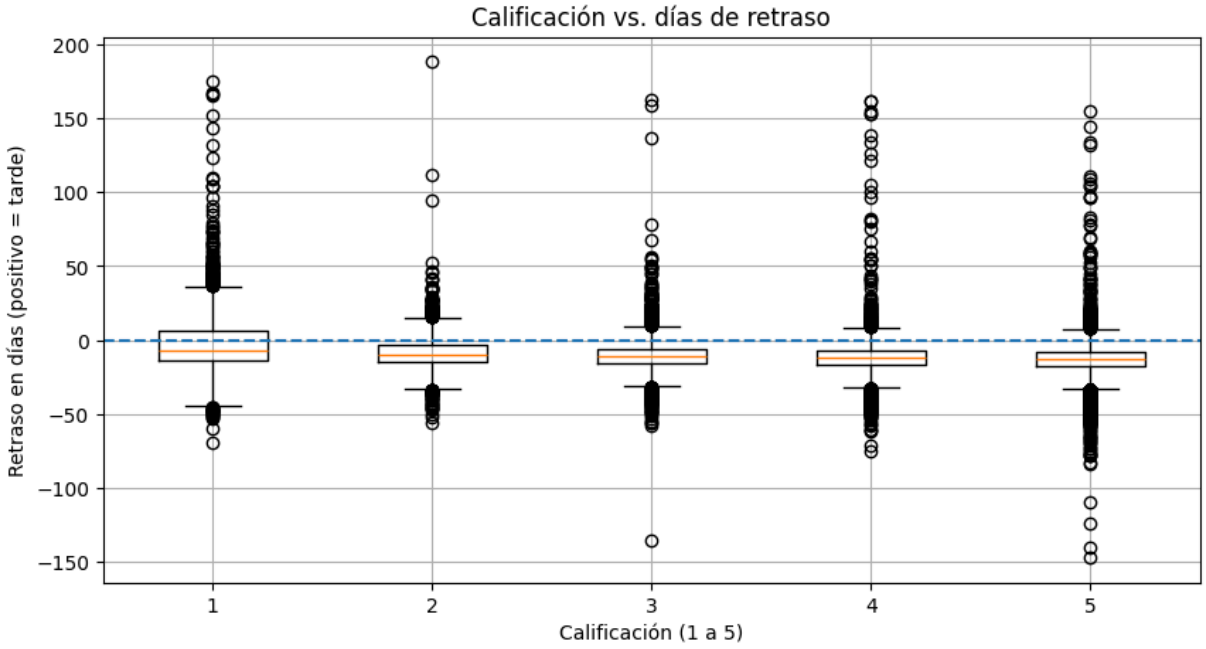
datos_boxplot = []
etiquetas = []

for calificacion in [1, 2, 3, 4, 5]:
    datos = retraso_resena.loc[
        retraso_resena["review_score"] == calificacion, "retraso"
```

```
    ].dropna()
    datos_boxplot.append(datos.values)
    etiquetas.append(str(calificacion))

plt.boxplot(datos_boxplot, labels=etiquetas)
plt.axhline(0, linestyle="--")
plt.title("Calificación vs. días de retraso")
plt.xlabel("Calificación (1 a 5)")
plt.ylabel("Retraso en días (positivo = tarde)")
plt.show()
```

```
/var/folders/7_/p0ymq6x90m95vyj7kx46t1sr0000gn/T/ipykernel_3613/2615778260.py:25: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
  plt.boxplot(datos_boxplot, labels=etiquetas)
```



6. Barras con error — calificación promedio a tiempo vs. tarde

Usamos la misma tabla de retrasos y reseñas. La desviación estándar se representa con `yerr`.

In [7]:
```
retraso_resena["tarde"] = retraso_resena["retraso"] > 0

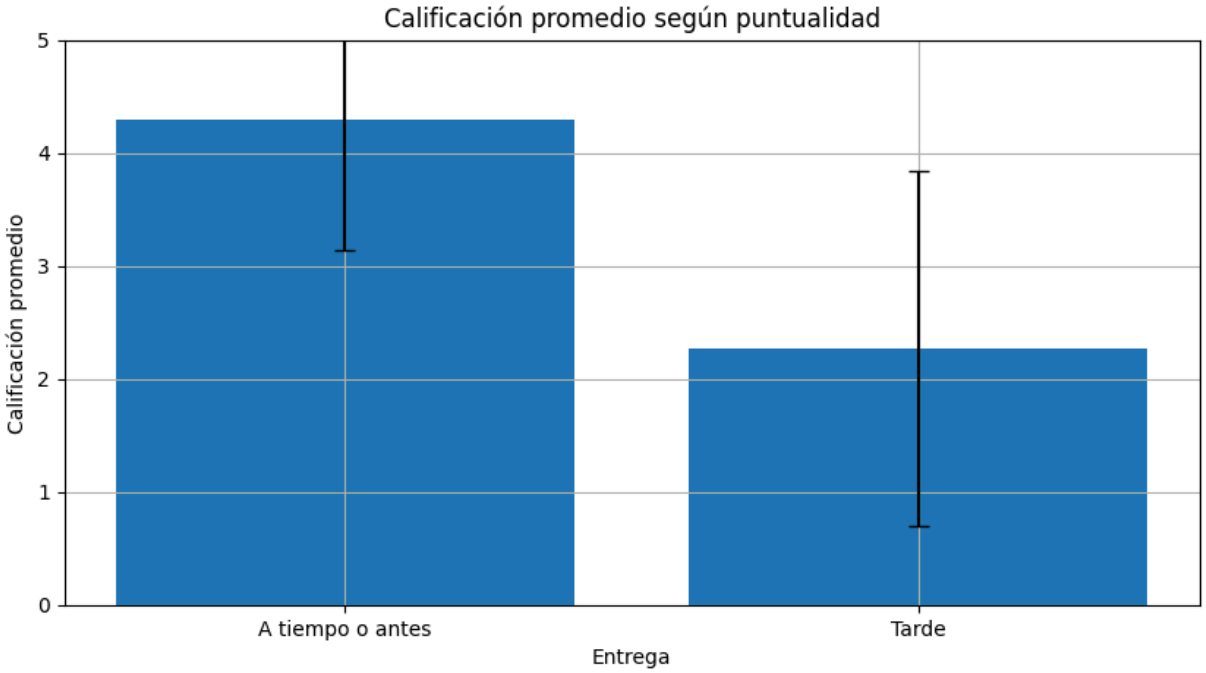
resumen_tarde = retraso_resena.groupby("tarde")["review_score"].agg(["mean",
resumen_tarde = resumen_tarde.reindex([False, True])

medias = resumen_tarde["mean"].values
errores = resumen_tarde["std"].fillna(0).values

plt.bar(["A tiempo o antes", "Tarde"], medias, yerr=errores, capsize=5)
plt.title("Calificación promedio según puntualidad")
plt.xlabel("Entrega")
```

```
plt.ylabel("Calificación promedio")
plt.ylim(0, 5)
plt.show()

print(resumen_tarde)
```




```
           mean       std
tarde
False  4.290582  1.149084
True   2.271702  1.571613
```

7. Pastel y dona — métodos de pago

Contamos las filas de cada método de pago. Para este gráfico, cada fila representa un registro de pago, no necesariamente un pedido único. Agrupamos los métodos poco frecuentes en “Otros”.

In [8]:
```
pagos = order_payments["payment_type"].fillna("not_defined").value_counts()
total_pagos = pagos.sum()
porcentajes = pagos / total_pagos

principales = porcentajes[porcentajes >= 0.03]
otros = porcentajes[porcentajes < 0.03].sum()

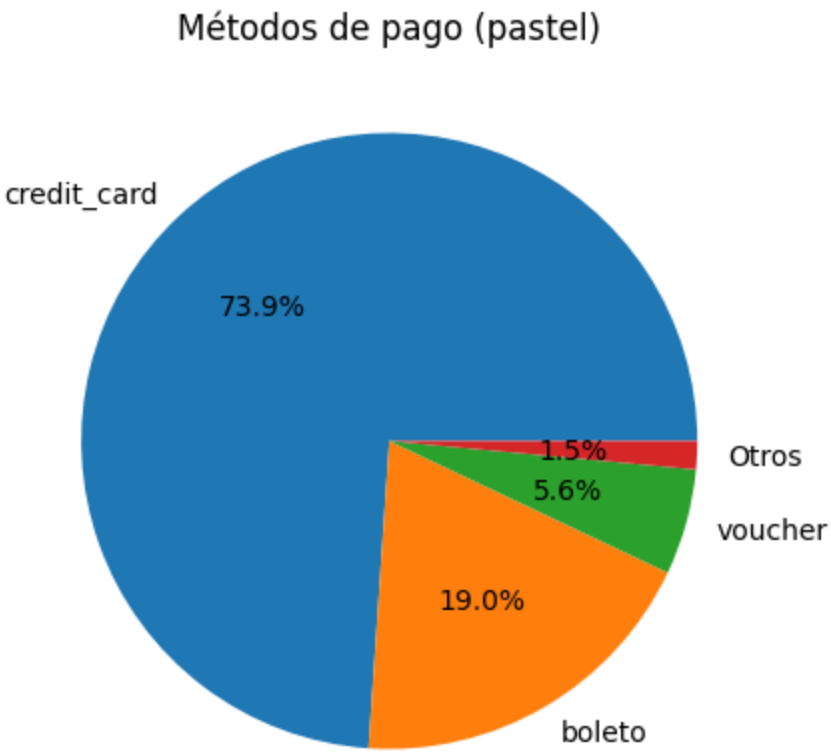
if otros > 0:
    principales["Otros"] = otros

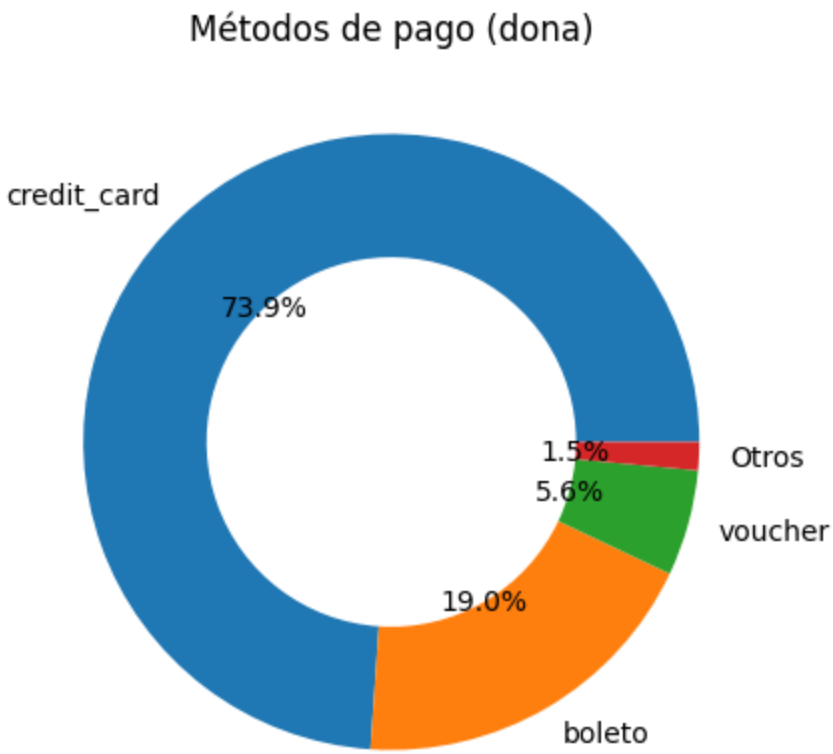
plt.pie(principales.values, labels=principales.index, autopct="%1.1f%%")
plt.title("Métodos de pago (pastel)")
plt.show()

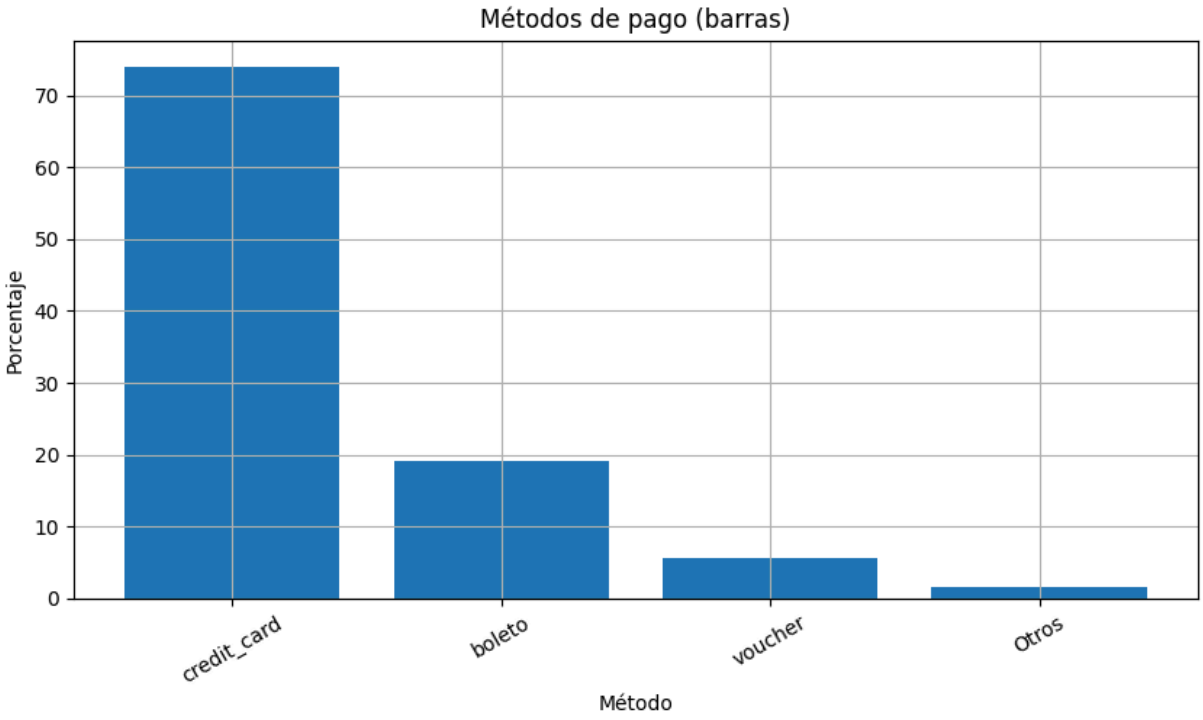
plt.pie(
    principales.values,
    labels=principales.index,
```

```
    autopct="%1.1f%%",
    wedgeprops={"width": 0.4}
)
plt.title("Métodos de pago (dona)")
plt.show()

plt.bar(principales.index, principales.values * 100)
plt.title("Métodos de pago (barras)")
plt.xlabel("Método")
plt.ylabel("Porcentaje")
plt.xticks(rotation=30)
plt.show()
```







8. Barras agrupadas — monto promedio por mensualidades y método de pago

Creamos grupos de mensualidades con pd.cut, calculamos el promedio de payment_value y organizamos el resultado con pivot_table.

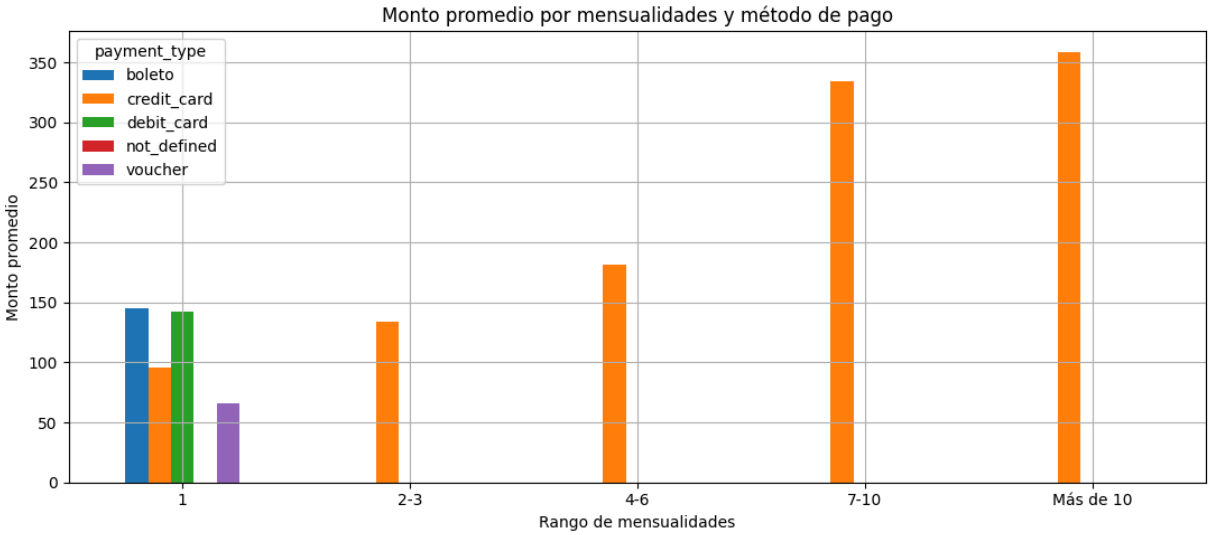
In [9]:
```
pagos_meses = order_payments.copy()

pagos_meses["rango_mensualidades"] = pd.cut(
    pagos_meses["payment_installments"],
    bins=[0, 1, 3, 6, 10, np.inf],
    labels=["1", "2-3", "4-6", "7-10", "Más de 10"]
)

tabla_pagos = pagos_meses.pivot_table(
    index="rango_mensualidades",
    columns="payment_type",
    values="payment_value",
    aggfunc="mean",
    observed=False
)

ax = tabla_pagos.plot(kind="bar", figsize=(11, 5))
ax.set_title("Monto promedio por mensualidades y método de pago")
ax.set_xlabel("Rango de mensualidades")
ax.set_ylabel("Monto promedio")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

tabla_pagos
```



Out[9]:

payment_type	boleto	credit_card	debit_card	not_defined	voucher
rango_mensualidades					
1	145.034435	95.872930	142.57017	0.0	65.703354
2-3	NaN	134.230429	NaN	NaN	NaN
4-6	NaN	181.318430	NaN	NaN	NaN
7-10	NaN	333.825551	NaN	NaN	NaN
Más de 10	NaN	358.344839	NaN	NaN	NaN

9. Barras apiladas — estado del pedido por estado de la república

Unimos pedidos con clientes por customer_id. La tabla dinámica cuenta pedidos por estado y estatus. Después dividimos cada fila entre su total para mostrar porcentajes.

In [10]:
```
orders_customers = orders.merge(customers, on="customer_id", how="left")

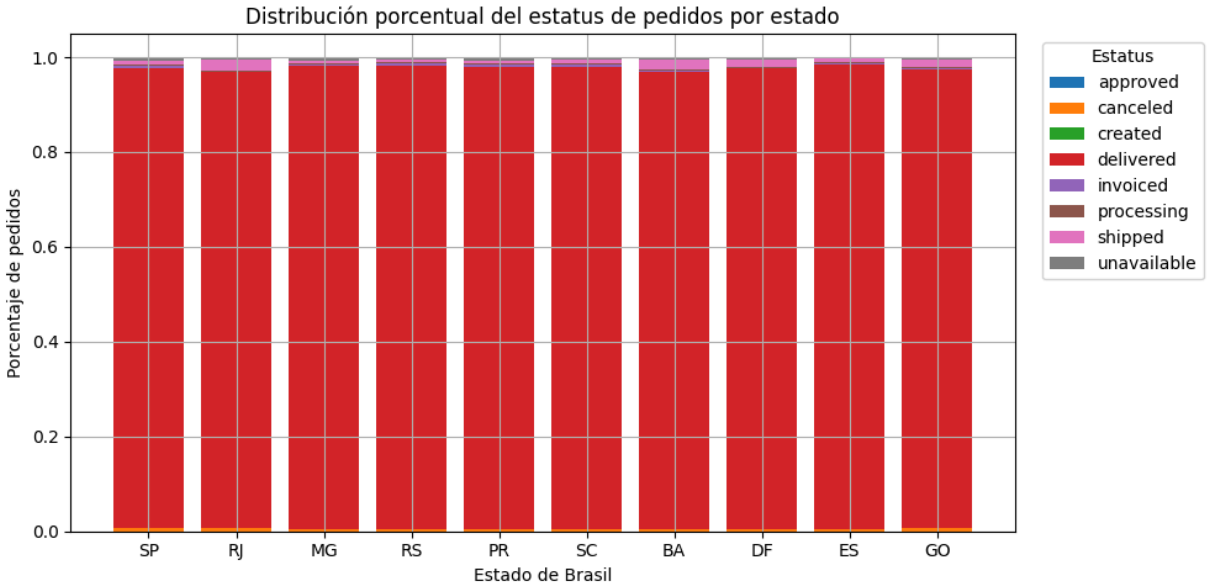
tabla_estados = orders_customers.pivot_table(
    index="customer_state",
    columns="order_status",
    values="order_id",
    aggfunc="count",
    fill_value=0
)

tabla_porcentaje = tabla_estados.div(tabla_estados.sum(axis=1), axis=0)
top_estados = tabla_estados.sum(axis=1).nlargest(10).index
tabla_porcentaje = tabla_porcentaje.loc[top_estados]

bottom = np.zeros(len(tabla_porcentaje))
x = np.arange(len(tabla_porcentaje))

for estado_pedido in tabla_porcentaje.columns:
    valores = tabla_porcentaje[estado_pedido].values
    plt.bar(x, valores, bottom=bottom, label=estado_pedido)
    bottom = bottom + valores

plt.title("Distribución porcentual del estatus de pedidos por estado")
plt.xlabel("Estado de Brasil")
plt.ylabel("Porcentaje de pedidos")
plt.xticks(x, tabla_porcentaje.index)
plt.legend(title="Estatus", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()
```



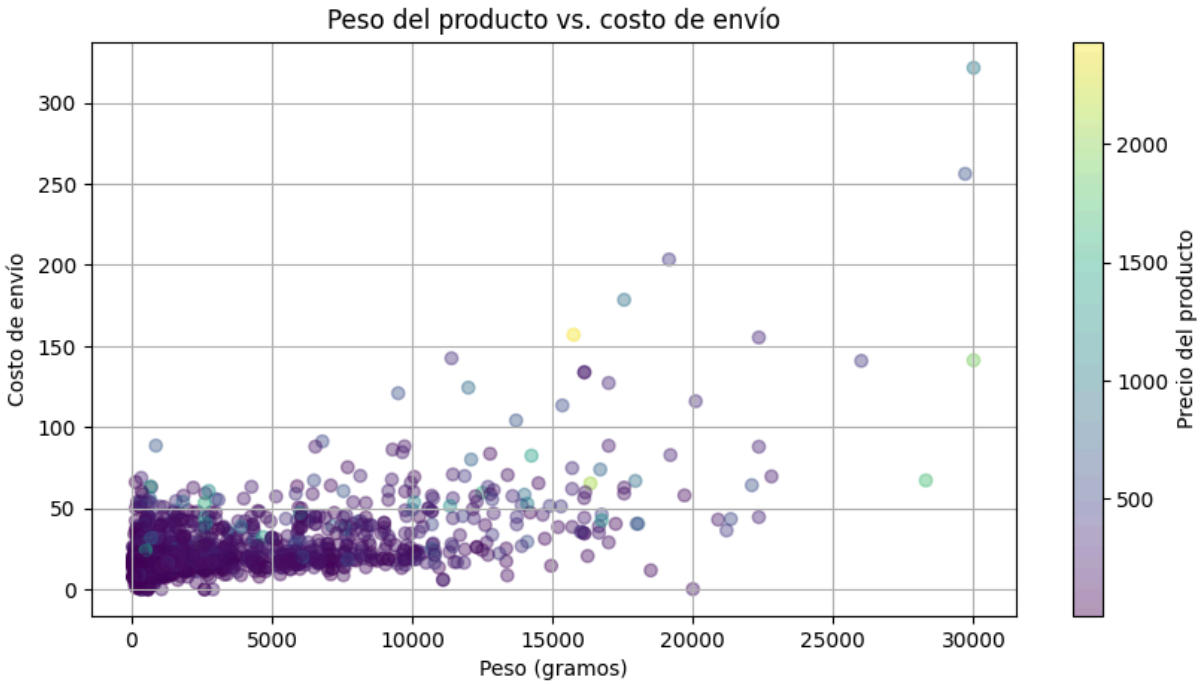
10. Dispersión — peso del producto vs. costo de envío

Unimos artículos con productos, quitamos datos faltantes y tomamos una muestra de hasta 3,000 filas para evitar saturar la gráfica. El color representa el precio.

In [11]:
```
peso_envio = order_items.merge(products, on="product_id", how="left")
peso_envio = peso_envio.dropna(
    subset=["product_weight_g", "freight_value", "price"]
)
peso_envio = peso_envio[
    (peso_envio["product_weight_g"] > 0) &
    (peso_envio["freight_value"] >= 0)
]

muestra = peso_envio.sample(n=min(3000, len(peso_envio)), random_state=42)

puntos = plt.scatter(
    muestra["product_weight_g"],
    muestra["freight_value"],
    c=muestra["price"],
    alpha=0.4
)
plt.title("Peso del producto vs. costo de envío")
plt.xlabel("Peso (gramos)")
plt.ylabel("Costo de envío")
plt.colorbar(puntos, label="Precio del producto")
plt.show()
```



11. Mapa de calor — pedidos por día de la semana y hora

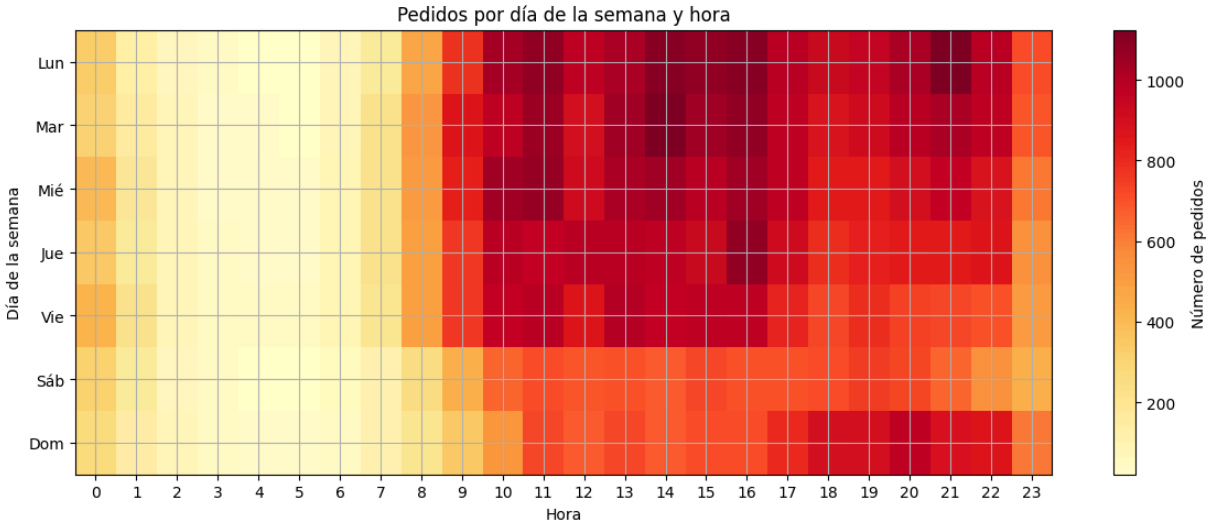
Extraemos día de la semana y hora de compra. day of week usa lunes = 0 y domingo = 6.

El mapa de calor ayuda a identificar las horas y días con mayor actividad.

In [12]:
```
fechas = orders.dropna(subset=["order_purchase_timestamp"]).copy()
fechas["dia_semana"] = fechas["order_purchase_timestamp"].dt.dayofweek
fechas["hora"] = fechas["order_purchase_timestamp"].dt.hour

calor = fechas.pivot_table(
    index="dia_semana",
    columns="hora",
    values="order_id",
    aggfunc="count",
    fill_value=0
).reindex(index=range(7), columns=range(24), fill_value=0)

fig, ax = plt.subplots(figsize=(12, 5))
imagen = ax.imshow(calor.values, aspect="auto", cmap="YlOrRd")
ax.set_title("Pedidos por día de la semana y hora")
ax.set_xlabel("Hora")
ax.set_ylabel("Día de la semana")
ax.set_xticks(range(24))
ax.set_yticks(range(7))
ax.set_yticklabels(["Lun", "Mar", "Mié", "Jue", "Vie", "Sáb", "Dom"])
fig.colorbar(imagen, ax=ax, label="Número de pedidos")
plt.tight_layout()
plt.show()
```



12. Dashboard final — subplots y doble eje

Esta gráfica reutiliza los resultados anteriores:

- Panel superior izquierdo: pedidos por mes y facturación de productos por mes.
- Panel superior derecho: top 10 categorías por ingreso.
- Panel inferior izquierdo: retraso por calificación.
- Panel inferior derecho: mapa de calor por día y hora.

Para el ingreso mensual, unimos pedidos con artículos. Como cada pedido puede tener varios artículos, sumar el price después de este merge es correcto para calcular el ingreso por artículos; no se agregan pagos en esta tabla.

In [13]:
```
# Datos mensuales de pedidos
pedidos_dashboard = pedidos_mes.copy()

# Ingreso mensual: unir orders con order_items y sumar el precio de cada art
ordenes_con_items = orders.merge(
    order_items[["order_id", "price"]], on="order_id", how="inner"
)
ordenes_con_items["mes"] = ordenes_con_items["order_purchase_timestamp"].dt.
ingreso_mes = ordenes_con_items.groupby("mes")["price"].sum()

# Usar los mismos meses que el gráfico de pedidos
ingreso_mes = ingreso_mes.reindex(pedidos_dashboard.index)

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Panel 1: pedidos e ingreso mensual con doble eje
ax1 = axes[0, 0]
meses = pedidos_dashboard.index.astype(str)
ax1.plot(meses, pedidos_dashboard.values, marker="o", label="Pedidos")
ax1.set_title("Pedidos e ingreso por mes")
ax1.set_xlabel("Mes")
ax1.set_ylabel("Pedidos")
```

```
ax1.tick_params(axis="x", rotation=60)

ax1b = ax1.twinx()
ax1b.plot(meses, ingreso_mes.values, color="orange", marker=".", label="Ingr
ax1b.set_ylabel("Ingreso por artículos")

# Leyenda combinada
lineas1, etiquetas1 = ax1.get_legend_handles_labels()
lineas2, etiquetas2 = ax1b.get_legend_handles_labels()
ax1.legend(lineas1 + lineas2, etiquetas1 + etiquetas2, loc="upper left")

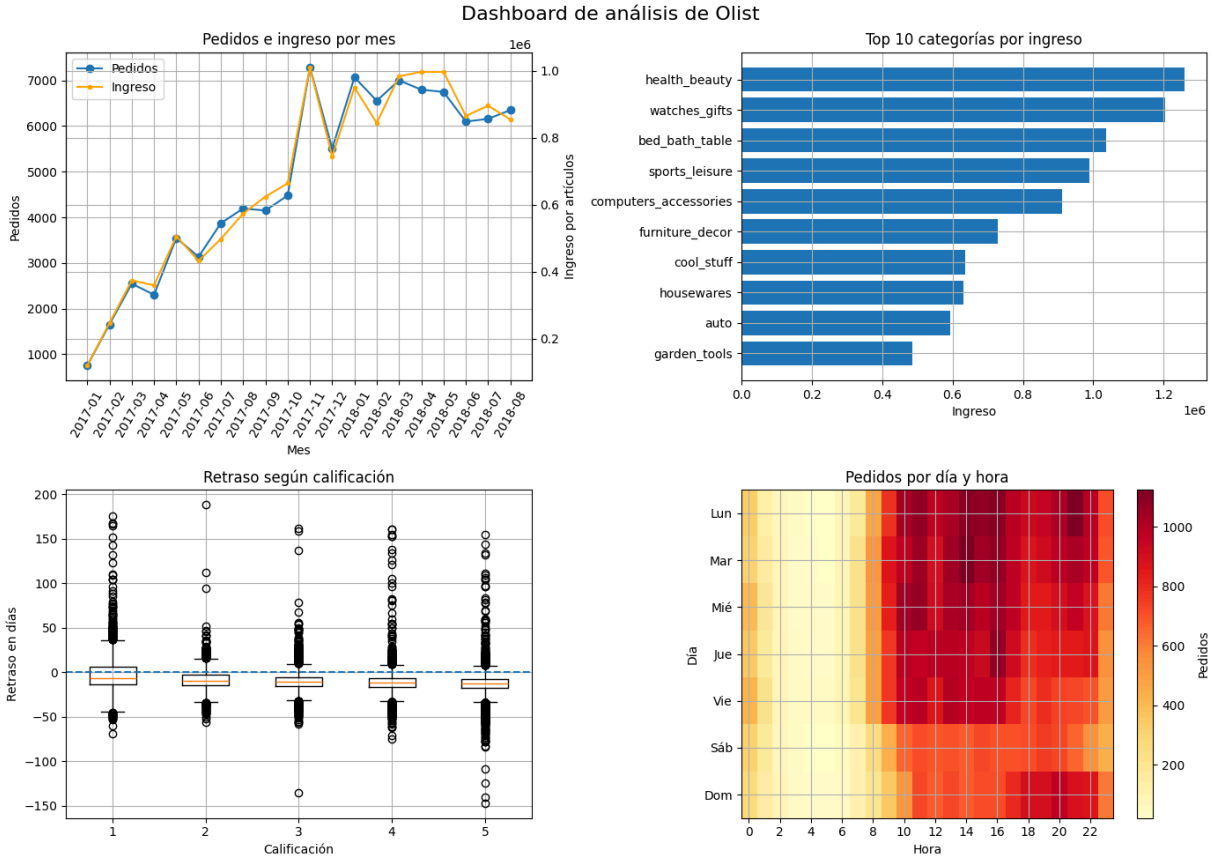
# Panel 2: top 10 categorías
axes[0, 1].barh(top10.index, top10.values)
axes[0, 1].set_title("Top 10 categorías por ingreso")
axes[0, 1].set_xlabel("Ingreso")

# Panel 3: boxplot de retraso por calificación
axes[1, 0].boxplot(datos_boxplot, labels=etiquetas)
axes[1, 0].axhline(0, linestyle="--")
axes[1, 0].set_title("Retraso según calificación")
axes[1, 0].set_xlabel("Calificación")
axes[1, 0].set_ylabel("Retraso en días")

# Panel 4: heatmap
imagen = axes[1, 1].imshow(calor.values, aspect="auto", cmap="YlOrRd")
axes[1, 1].set_title("Pedidos por día y hora")
axes[1, 1].set_xlabel("Hora")
axes[1, 1].set_ylabel("Día")
axes[1, 1].set_xticks(range(0, 24, 2))
axes[1, 1].set_yticks(range(7))
axes[1, 1].set_yticklabels(["Lun", "Mar", "Mié", "Jue", "Vie", "Sáb", "Dom"]
fig.colorbar(imagen, ax=axes[1, 1], label="Pedidos")

fig.suptitle("Dashboard de análisis de Olist", fontsize=16)
plt.tight_layout()
plt.savefig("dashboard.png", dpi=150, bbox_inches="tight")
plt.show()
```

```
/var/folders/7_/p0ymq6x90m95vyj7kx46t1sr0000gn/T/ipykernel_3613/576928723.p
y:40: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has
been renamed 'tick_labels' since Matplotlib 3.9; support for the old name wi
ll be dropped in 3.11.
  axes[1, 0].boxplot(datos_boxplot, labels=etiquetas)
```



Comentarios finales: trampas importantes

1. **Granularidad:** `order_items` tiene varias filas por pedido y `order_payments` también puede tener varias. Si unes ambas tablas entre sí, los montos pueden multiplicarse.
2. **Reseñas duplicadas:** un pedido puede tener más de una fila en `order_reviews`. Para este ejercicio usamos `drop_duplicates` por `order_id`.
3. **Fechas:** conviértelas con `pd.to_datetime` antes de usar `.dt`.
4. **Categorías:** los nombres originales están en portugués; la tabla de traducción permite usar nombres en inglés.
5. **Datos extremos:** antes de eliminar datos o recortar fechas, revisa las cantidades y justifica el criterio.
6. **Interpretación:** una gráfica muestra patrones, pero no demuestra por sí sola que una variable cause otra.